



Java Concurrency & Multi-threading: Interview Guide

Table of Contents

Fundamentals of Threading	4
Q.1 What is the difference between a process and a thread? How does the JVM handle threads?	4
Q.2 Explain the thread lifecycle in Java. What are the different states a thread can be in?	5
Q.3 What are the different ways to create a thread in Java? Compare extending Thread class vs implementing Runnable interface.	6
Q.4 What is the purpose of the volatile keyword in Java? How does it differ from synchronized?	7
Q.5 Explain the difference between wait() and sleep() methods. When would you use each? ..	8
Q.6 What is thread safety? How do you achieve it in Java?	9
Q.7 What is a race condition? Provide an example and how you would prevent it.	10
Q.8 Explain the concept of thread synchronization. What problems does it solve?	11
Q.9 What is a deadlock? What conditions must be present for a deadlock to occur?	12
Q.10 What is the difference between notify() and notifyAll()? When should you use each? ..	14
Q.11 Explain the synchronized keyword. Can you synchronize on a primitive type?	15
Q.12 What is thread starvation and how can you prevent it?	17
Q.13 What is the purpose of the join() method in threads?	18
Q.14 What are daemon threads? How do they differ from user threads?	20
Q.15 What is the difference between concurrent and parallel execution?	21
Basic Concurrency Utilities	24
Q.16 What is the Executor framework? Why is it preferred over manually creating threads? ..	24
Q.17 Explain the difference between Callable and Runnable interfaces.	25
Q.18 What is a Future in Java? How do you use it to get results from asynchronous tasks? ...	27
Q.19 Describe the different types of thread pools available in Executors class.	28
Q.20 What is ThreadLocal? When would you use it and what are its potential pitfalls?	29
Q.21 What is the difference between execute() and submit() methods in ExecutorService? ..	30
Q.22 Explain CountdownLatch. Provide a use case where it would be beneficial.	31
Q.23 What is a CyclicBarrier? How does it differ from CountdownLatch?	33

Q.24 What is a Semaphore? Explain with an example use case.	34
Q.25 What are the common thread-safe collections in Java? Name at least five.	36
Q.26 Explain ConcurrentHashMap. How does it achieve better performance than Hashtable?	37
Q.27 What is CopyOnWriteArrayList? When should you use it?	37
Q.28 What is the difference between BlockingQueue and regular Queue?	38
Q.29 What is an AtomicInteger? How does it differ from using synchronized on an integer? .	39
Advanced Concurrency Patterns	42
Q.30 Explain the happens-before relationship in the Java Memory Model. Why is it important?	42
Q.31 What is the difference between optimistic and pessimistic locking? Provide examples of each in Java.	42
Q.32 Describe the Fork/Join framework. When would you use it over traditional thread pools?	43
Q.33 What is work stealing in the context of Fork/Join? How does it improve performance? .	44
Q.34 Explain the concept of lock striping. How does ConcurrentHashMap use this technique?	45
Q.35 What are reentrant locks? How do they differ from intrinsic locks (synchronized)?.....	45
Q.36 Describe read-write locks. When would ReentrantReadWriteLock be more efficient than synchronized?.....	47
Q.37 What is a StampedLock? How does it improve upon ReentrantReadWriteLock?.....	48
Q.38 Explain the ABA problem in concurrent programming. How would you detect or prevent it?	49
Q.39 What is compare-and-swap (CAS)? How do atomic classes use it?	50
Q.40 Describe the concept of lock-free and wait-free algorithms. Provide an example.....	51
Q.41 Explain how CompletableFuture improves upon Future. Provide examples of its advanced usage.	52
Q.42 What is reactive programming? How does it relate to concurrency in Java?	53
Q.43 What tools would you use to detect and diagnose deadlocks in a production Java application?	55

Q.44 What is the overhead of synchronization? How would you measure it in your application?	55
Q.45 Explain how garbage collection can impact multi-threaded application performance..	56
Q.46 What is livelock? How does it differ from deadlock and how do you resolve it?	57
Q.47 How would you implement a timeout mechanism for acquiring locks to prevent indefinite blocking?	59

Fundamentals of Threading

Q.1 What is the difference between a process and a thread? How does the JVM handle threads?

Process is an independent program in execution with its own memory space. Each process has separate heap, stack, and code segments isolated from other processes.

Thread is a lightweight unit of execution within a process. Threads share the same memory space (heap) but have their own stack and program counter.

JVM Thread Handling: The JVM maps Java threads to native OS threads (1:1 mapping in most implementations). The JVM's Thread Scheduler works with the OS scheduler to allocate CPU time. The JVM manages thread states, synchronization primitives, and memory visibility through the Java Memory Model.

```
// Example: Process vs Thread concept
public class ProcessVsThreadExample {
    private static int sharedCounter = 0; // Shared among threads

    public static void main(String[] args) {
        // Creating two threads in same process
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                sharedCounter++; // Both threads can access this
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                sharedCounter++;
            }
        });

        t1.start();
        t2.start();
    }
}
```

Q.2 Explain the thread lifecycle in Java. What are the different states a thread can be in?

A thread goes through several states during its lifetime:

NEW: Thread is created but not yet started.

RUNNABLE: Thread is executing or ready to execute, waiting for CPU allocation.

BLOCKED: Thread is waiting to acquire a monitor lock to enter a synchronized block/method.

WAITING: Thread is waiting indefinitely for another thread to perform a specific action (using `wait()`, `join()`, or `LockSupport.park()`).

TIMED_WAITING: Thread is waiting for a specified time period (using `sleep()`, `wait(timeout)`, `join(timeout)`).

TERMINATED: Thread has completed execution or was terminated abnormally.

```
public class ThreadLifecycleExample {
    public static void main(String[] args) throws InterruptedException {
        Thread thread = new Thread(() -> {
            try {
                System.out.println("Thread running: " + Thread.currentThread().getState());
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        System.out.println("NEW state: " + thread.getState()); // NEW

        thread.start();
        System.out.println("After start: " + thread.getState()); // RUNNABLE

        Thread.sleep(100);
        System.out.println("While sleeping: " + thread.getState()); // TIMED_WAITING

        thread.join();
        System.out.println("TERMINATED state: " + thread.getState()); // TERMINATED
    }
}
```

Q.3 What are the different ways to create a thread in Java? Compare extending Thread class vs implementing Runnable interface.

There are multiple ways to create threads in Java:

- 1. Extending Thread class:** Simple but limits inheritance since Java doesn't support multiple inheritance.
- 2. Implementing Runnable interface:** Preferred approach as it allows the class to extend another class and promotes better object-oriented design by separating the task from the thread mechanism.
- 3. Implementing Callable interface:** Similar to Runnable but can return a result and throw checked exceptions.
- 4. Using Executor framework:** Modern, recommended approach for managing thread pools.

```
// Method 1: Extending Thread class
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread using Thread class: " + getName());
    }
}

// Method 2: Implementing Runnable interface (Preferred)
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Thread using Runnable: " + Thread.currentThread().getName());
    }
}

public class ThreadCreationExample {
    public static void main(String[] args) {
        // Using Thread class
        MyThread t1 = new MyThread();
        t1.start();

        // Using Runnable interface
        Thread t2 = new Thread(new MyRunnable());
        t2.start();

        // Using lambda (since Runnable is functional interface)
        Thread t3 = new Thread(() ->
            System.out.println("Lambda thread: " + Thread.currentThread().getName())
        );
        t3.start();
    }
}
```

Comparison: Runnable is preferred because it allows class flexibility, promotes composition over inheritance, enables the same Runnable instance to be used by multiple threads, and works seamlessly with executor services.

Q.4 What is the purpose of the volatile keyword in Java? How does it differ from synchronized?

volatile ensures that changes to a variable are immediately visible to all threads. It prevents caching of variables in thread-local memory and ensures happens-before relationship.

Key differences from synchronized: volatile only ensures visibility, not atomicity. It works on individual variables, while synchronized works on blocks/methods. volatile cannot prevent race conditions for compound operations (like increment), whereas synchronized provides mutual exclusion and prevents race conditions. volatile has lower overhead but limited functionality.

```
public class VolatileExample {
    private volatile boolean running = true; // Ensures visibility
    private int counter = 0; // Not thread-safe even with volatile

    public void stop() {
        running = false; // Change visible to all threads immediately
    }

    public void increment() {
        counter++; // NOT atomic, even if counter were volatile
    }

    // Synchronized ensures both visibility and atomicity
    public synchronized void safeIncrement() {
        counter++;
    }

    public static void main(String[] args) throws InterruptedException {
        VolatileExample example = new VolatileExample();

        Thread worker = new Thread(() -> {
            while (example.running) {
                // Do work
            }
            System.out.println("Worker stopped");
        });

        worker.start();
        Thread.sleep(1000);
        example.stop(); // This change will be visible immediately
        worker.join();
    }
}
```


Q.5 Explain the difference between wait() and sleep() methods. When would you use each?

wait() is defined in Object class, releases the monitor lock, must be called within a synchronized context, thread can be awakened by notify() or notifyAll(), and used for inter-thread communication.

sleep() is defined in Thread class, does NOT release any locks held, can be called from anywhere, thread wakes up after specified time (unless interrupted), and used for pausing execution.

```
public class WaitVsSleepExample {
    private static final Object lock = new Object();

    public static void main(String[] args) {
        // wait() example - releases lock
        Thread t1 = new Thread(() -> {
            synchronized (lock) {
                try {
                    System.out.println("T1: Waiting...");
                    lock.wait(); // Releases lock, waits for notification
                    System.out.println("T1: Resumed after notification");
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });

        // sleep() example - holds lock
        Thread t2 = new Thread(() -> {
            synchronized (lock) {
                try {
                    System.out.println("T2: Sleeping...");
                    Thread.sleep(2000); // Holds lock during sleep
                    System.out.println("T2: Woke up, notifying");
                    lock.notify(); // Wake up waiting thread
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });

        t1.start();
        try { Thread.sleep(100); } catch (InterruptedException e) {}
        t2.start();
    }
}
```

Q.6 What is thread safety? How do you achieve it in Java?

Thread safety means a class or method behaves correctly when accessed by multiple threads simultaneously, regardless of scheduling or interleaving of execution, without requiring external synchronization.

Ways to achieve thread safety:

1. Synchronization using synchronized keyword
2. Using volatile for visibility guarantees
3. Using atomic classes from java.util.concurrent.atomic
4. Using concurrent collections from java.util.concurrent
5. Using immutable objects
6. Thread-local variables using ThreadLocal
7. Using locks from java.util.concurrent.locks

```
import java.util.concurrent.atomic.AtomicInteger;

// NOT thread-safe
class UnsafeCounter {
    private int count = 0;
    public void increment() { count++; }
    public int getCount() { return count; }
}

// Thread-safe using synchronization
class SynchronizedCounter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public synchronized int getCount() { return count; }
}

// Thread-safe using atomic
class AtomicCounter {
    private AtomicInteger count = new AtomicInteger(0);
    public void increment() { count.incrementAndGet(); }
    public int getCount() { return count.get(); }
}

// Thread-safe using immutability
final class ImmutableCounter {
    private final int count;
    public ImmutableCounter(int count) { this.count = count; }
    public ImmutableCounter increment() { return new ImmutableCounter(count + 1); }
    public int getCount() { return count; }
}
```

Q.7 What is a race condition? Provide an example and how you would prevent it.

A **race condition** occurs when two or more threads access shared data concurrently and at least one thread modifies it, leading to unpredictable results. The outcome depends on the timing and interleaving of thread execution.

```
public class RaceConditionExample {
    private int balance = 1000;

    // Race condition - NOT thread-safe
    public void unsafeWithdraw(int amount) {
        if (balance >= amount) { // Check
            // Context switch can happen here!
            balance -= amount; // Modify
            System.out.println("Withdrawn: " + amount + ", Balance: " + balance);
        }
    }

    // Solution 1: Synchronized method
    public synchronized void safeWithdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount + ", Balance: " + balance);
        }
    }

    // Solution 2: Synchronized block
    public void withdrawWithBlock(int amount) {
        synchronized (this) {
            if (balance >= amount) {
                balance -= amount;
                System.out.println("Withdrawn: " + amount + ", Balance: " + balance);
            }
        }
    }

    public static void main(String[] args) {
        RaceConditionExample account = new RaceConditionExample();

        // Multiple threads trying to withdraw
        Runnable task = () -> account.unsafeWithdraw(600);

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();
        // Both might succeed even though balance is only 1000!
    }
}
```

Prevention strategies: Use synchronized blocks/methods, use atomic variables, use concurrent data structures, use explicit locks (ReentrantLock), design immutable objects, and minimize shared mutable state.

Q.8 Explain the concept of thread synchronization. What problems does it solve?

Thread synchronization is a mechanism to control access to shared resources by multiple threads to prevent data inconsistency and race conditions.

Problems it solves:

1. Race conditions
2. Data inconsistency
3. Memory visibility issues
4. Lost updates
5. Compound operation atomicity

Java provides synchronization through the synchronized keyword, which works with intrinsic locks (monitor locks). Every object has an associated monitor lock that only one thread can hold at a time.

```
public class SynchronizationExample {
    private int count = 0;
    private final Object lock = new Object();

    // Method-level synchronization (locks on 'this')
    public synchronized void synchronizedMethod() {
        count++;
    }

    // Block-level synchronization (locks on 'this')
    public void synchronizedBlock() {
        synchronized (this) {
            count++;
        }
    }

    // Custom lock object
    public void customLock() {
        synchronized (lock) {
            count++;
        }
    }

    // Static synchronization (locks on Class object)
```

```

public static synchronized void staticSync() {
    // Locks on SynchronizationExample.class
}

public static void main(String[] args) throws InterruptedException {
    SynchronizationExample example = new SynchronizationExample();

    Runnable task = () -> {
        for (int i = 0; i < 1000; i++) {
            example.synchronizedMethod();
        }
    };

    Thread t1 = new Thread(task);
    Thread t2 = new Thread(task);

    t1.start();
    t2.start();

    t1.join();
    t2.join();

    System.out.println("Final count: " + example.count); // Will be 2000
}
}

```

Q.9 What is a deadlock? What conditions must be present for a deadlock to occur?

Deadlock is a situation where two or more threads are permanently blocked, each waiting for a resource held by another thread in the cycle.

Four necessary conditions for deadlock (Coffman conditions):

1. **Mutual Exclusion:** At least one resource must be held in non-shareable mode
2. **Hold and Wait:** A thread holds at least one resource and waits for additional resources
3. **No Preemption:** Resources cannot be forcibly taken from threads
4. **Circular Wait:** A circular chain of threads exists where each thread waits for a resource held by the next thread in the chain

Prevention strategies: Use consistent lock ordering, use lock timeout (tryLock with timeout), avoid nested locks when possible, use higher-level concurrency utilities, and detect and recover from deadlocks.

```

public class DeadlockExample {
    private final Object lock1 = new Object();
    private final Object lock2 = new Object();

    // Thread 1 execution path
    public void method1() {
        synchronized (lock1) {
            System.out.println(Thread.currentThread().getName() + ": Holding lock1...");
            try { Thread.sleep(100); } catch (InterruptedException e) {}

            System.out.println(Thread.currentThread().getName() + ": Waiting for
lock2...");
            synchronized (lock2) {
                System.out.println(Thread.currentThread().getName() + ": Holding lock1 &
lock2");
            }
        }
    }

    // Thread 2 execution path - DEADLOCK: Different lock order!
    public void method2() {
        synchronized (lock2) {
            System.out.println(Thread.currentThread().getName() + ": Holding lock2...");
            try { Thread.sleep(100); } catch (InterruptedException e) {}

            System.out.println(Thread.currentThread().getName() + ": Waiting for
lock1...");
            synchronized (lock1) {
                System.out.println(Thread.currentThread().getName() + ": Holding lock2 &
lock1");
            }
        }
    }

    // SOLUTION: Always acquire locks in same order
    public void safMethod1() {
        synchronized (lock1) {
            synchronized (lock2) {
                // Safe - consistent lock ordering
            }
        }
    }

    public void safeMethod2() {
        synchronized (lock1) { // Same order as safeMethod1
            synchronized (lock2) {
                // Safe - consistent lock ordering
            }
        }
    }
}

```

```

public static void main(String[] args) {
    DeadlockExample example = new DeadlockExample();

    Thread t1 = new Thread(example::method1, "Thread-1");
    Thread t2 = new Thread(example::method2, "Thread-2");

    t1.start();
    t2.start();
    // Deadlock will occur!
}
}

```

Q.10 What is the difference between `notify()` and `notifyAll()`? When should you use each?

- **`notify()`** wakes up a single waiting thread (which one is not guaranteed). If multiple threads are waiting, only one is awakened.
- **`notifyAll()`** wakes up all waiting threads. All awakened threads compete for the lock, but only one will acquire it and proceed.

When to use:

- Use **`notify()`** when only one thread needs to proceed and all waiting threads are equivalent (same wait condition).
- Use **`notifyAll()`** when multiple threads might need to check their specific conditions, different threads wait for different conditions, or to avoid missed signals and ensure correctness (safer but slightly less efficient).

```

public class NotifyExample {
    private final Object lock = new Object();
    private boolean dataReady = false;

    // Producer
    public void produce() {
        synchronized (lock) {
            dataReady = true;
            System.out.println("Data produced");

            // notify() - wakes only one thread
            // lock.notify();

            // notifyAll() - wakes all waiting threads (safer)
            lock.notifyAll();
        }
    }
}

```

```

    }

    // Consumer
    public void consume() {
        synchronized (lock) {
            while (!dataReady) { // Always use while, not if
                try {
                    System.out.println(Thread.currentThread().getName() + " waiting...");
                    lock.wait();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
            System.out.println(Thread.currentThread().getName() + " consumed data");
        }
    }

    public static void main(String[] args) throws InterruptedException {
        NotifyExample example = new NotifyExample();

        // Multiple consumers
        Thread c1 = new Thread(example::consume, "Consumer-1");
        Thread c2 = new Thread(example::consume, "Consumer-2");
        Thread c3 = new Thread(example::consume, "Consumer-3");

        c1.start();
        c2.start();
        c3.start();

        Thread.sleep(1000);

        // Producer
        Thread p = new Thread(example::produce);
        p.start();

        // With notify(): only 1 consumer wakes up
        // With notifyAll(): all 3 consumers wake up (though only 1 proceeds at a time)
    }
}

```

Q.11 Explain the **synchronized** keyword. Can you synchronize on a primitive type?

The **synchronized** keyword provides mutual exclusion by acquiring an intrinsic lock on an object. It can be applied to methods or code blocks.

Can you synchronize on a primitive type? NO. You cannot synchronize on primitive types (int, boolean, char, etc.) because synchronization requires an object reference to acquire its monitor lock. Primitives are not objects.

If you need to synchronize based on a primitive value, you must use its wrapper class object or a dedicated lock object.

```
public class SynchronizedExample {
    private int counter = 0;

    // WRONG: Cannot synchronize on primitive
    // private int lock = 0;
    // synchronized (lock) {} // Compilation error

    // CORRECT: Synchronize on object
    private final Object lock = new Object();
    private Integer wrapperLock = 0; // NOT recommended - can change reference

    // Method synchronization
    public synchronized void increment() {
        counter++; // Synchronized on 'this'
    }

    // Block synchronization with explicit lock
    public void incrementWithBlock() {
        synchronized (lock) {
            counter++;
        }
    }

    // DANGER: Don't synchronize on wrapper that can change
    public void badExample() {
        synchronized (wrapperLock) { // BAD: reference can change
            counter++;
        }
        wrapperLock = 10; // Now we're synchronizing on different object!
    }

    // CORRECT: Use final lock object
    private final Integer finalWrapperLock = Integer.valueOf(1);
    public void betterExample() {
        synchronized (finalWrapperLock) {
            counter++;
        }
    }

    // Static synchronization
    public static synchronized void staticMethod() {
        // Synchronized on Class object: SynchronizedExample.class
    }
}
```

Best practices: Always synchronize on final object references to prevent the lock object from changing, prefer private lock objects to prevent external code from acquiring your locks, never synchronize on String literals (they are interned), and avoid synchronizing on Boolean.TRUE or Boolean.FALSE (shared across JVM).

Q.12 What is thread starvation and how can you prevent it?

Thread starvation occurs when a thread is perpetually denied access to resources it needs, typically CPU time, because other threads are monopolizing those resources. The starved thread makes no progress despite being runnable.

Common causes:

1. High-priority threads constantly running
2. Threads holding locks for too long
3. Inefficient synchronization
4. Poorly configured thread pools
5. Unfair lock acquisition

Prevention strategies:

1. Use fair locks (ReentrantLock with fairness parameter)
2. Avoid holding locks for extended periods
3. Use appropriate thread priorities (don't abuse HIGH priority)
4. Use proper thread pool configurations
5. Implement timeouts for lock acquisition
6. Monitor thread pool queue sizes
7. Avoid infinite loops in synchronized blocks

```
import java.util.concurrent.locks.ReentrantLock;

public class StarvationExample {
    // Unfair lock (default) - can cause starvation
    private final ReentrantLock unfairLock = new ReentrantLock();

    // Fair lock - prevents starvation
    private final ReentrantLock fairLock = new ReentrantLock(true);

    // Example of potential starvation
    public void unfairAccess() {
        unfairLock.lock();
    }
}
```

```

        try {
            // Long-running operation
            // High-priority threads might keep acquiring lock
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            unfairLock.unlock();
        }
    }

    // Prevents starvation with fair lock
    public void fairAccess() {
        fairLock.lock();
        try {
            // Fair lock ensures FIFO order
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            fairLock.unlock();
        }
    }

    public static void main(String[] args) {
        StarvationExample example = new StarvationExample();

        // Create threads with different priorities
        for (int i = 0; i < 5; i++) {
            Thread t = new Thread(example::fairAccess);
            t.setPriority(i % 2 == 0 ? Thread.MAX_PRIORITY : Thread.MIN_PRIORITY);
            t.start();
        }

        // With fair lock, even MIN_PRIORITY threads get access
        // With unfair lock, MAX_PRIORITY threads might dominate
    }
}

```

Q.13 What is the purpose of the join() method in threads?

The **join()** method causes the current thread to pause execution and wait for the thread on which join() is called to complete. It's used for thread coordination when you need to ensure one thread finishes before another continues.

Variants:

1. join() - waits indefinitely
2. join(long millis) - waits for specified milliseconds

3. join(long millis, int nanos) - waits with nanosecond precision

```

public class JoinExample {
    public static void main(String[] args) throws InterruptedException {
        Thread worker1 = new Thread(() -> {
            System.out.println("Worker 1 starting");
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println("Worker 1 completed");
        });

        Thread worker2 = new Thread(() -> {
            System.out.println("Worker 2 starting");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println("Worker 2 completed");
        });

        System.out.println("Main: Starting workers");
        worker1.start();
        worker2.start();

        System.out.println("Main: Waiting for worker1 to complete");
        worker1.join(); // Main thread waits here

        System.out.println("Main: Worker1 done, waiting for worker2");
        worker2.join(); // Main thread waits here

        System.out.println("Main: All workers completed, continuing");

        // Example with timeout
        Thread longRunner = new Thread(() -> {
            try {
                Thread.sleep(5000);
            } catch (InterruptedException e) {}
        });

        longRunner.start();
        longRunner.join(1000); // Wait max 1 second

        if (longRunner.isAlive()) {
            System.out.println("Thread still running after timeout");
        }
    }
}

```

Q.14 What are daemon threads? How do they differ from user threads?

User threads are high-priority threads that the JVM waits for before terminating. The JVM keeps running as long as at least one user thread is alive.

Daemon threads are low-priority background threads that provide services to user threads. The JVM doesn't wait for daemon threads to complete and exits when all user threads have finished, even if daemon threads are still running.

Key differences:

1. JVM termination behavior
2. Purpose (daemon for background services)
3. Default: threads inherit daemon status from creating thread
4. Must call `setDaemon(true)` before `start()`

Common uses for daemon threads: Garbage collection, finalizer thread, signal dispatchers, background monitoring, cleanup tasks, and heartbeat services.

```
public class DaemonThreadExample {
    public static void main(String[] args) throws InterruptedException {
        // User thread (default)
        Thread userThread = new Thread(() -> {
            for (int i = 0; i < 3; i++) {
                System.out.println("User thread running: " + i);
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
            System.out.println("User thread completed");
        });

        // Daemon thread
        Thread daemonThread = new Thread(() -> {
            while (true) {
                System.out.println("Daemon thread running (background service)");
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    break;
                }
            }
            System.out.println("Daemon thread completed"); // May not print
        });

        daemonThread.setDaemon(true); // MUST set before start()
```

```

        System.out.println("Is daemon? " + daemonThread.isDaemon()); // true
        System.out.println("Is user? " + userThread.isDaemon()); // false

        userThread.start();
        daemonThread.start();

        // JVM will wait for userThread to complete
        // JVM will NOT wait for daemonThread
        // Once userThread finishes, JVM exits, killing daemonThread

        // Output shows daemon thread gets interrupted mid-execution
    }
}

```

Important notes: You cannot change daemon status after thread starts, daemon threads should not perform I/O operations or access resources that need cleanup, and always check `thread.isDaemon()` if behavior depends on thread type.

Q.15 What is the difference between concurrent and parallel execution?

Concurrent execution means multiple tasks making progress during overlapping time periods. Tasks don't necessarily execute at the exact same instant but give the appearance of simultaneous execution through context switching. This is possible on a single-core processor.

Parallel execution means multiple tasks literally executing at the same instant on different cores or processors. It requires multiple processing units and is a subset of concurrency.

Key distinctions:

Concurrency is about dealing with multiple things at once (task structure), while parallelism is about doing multiple things at once (physical execution).

Concurrency is possible on single-core systems through time-slicing, while parallelism requires multi-core or multi-processor systems.

Concurrency improves responsiveness and resource utilization, while parallelism improves throughput and computational speed.

All parallel programs are concurrent, but not all concurrent programs are parallel.

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.stream.IntStream;

public class ConcurrentVsParallelExample {

```

```

// Concurrent execution example
public static void concurrentExample() {
    ExecutorService executor = Executors.newFixedThreadPool(2);

    for (int i = 0; i < 4; i++) {
        final int taskId = i;
        executor.submit(() -> {
            System.out.println("Concurrent task " + taskId +
                               " on thread: " + Thread.currentThread().getName());
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {}
        });
    }

    executor.shutdown();
    // 4 tasks on 2 threads - concurrent but not all parallel
}

// Parallel execution example
public static void parallelExample() {
    // Parallel stream uses ForkJoinPool
    IntStream.range(0, 4).parallel().forEach(i -> {
        System.out.println("Parallel task " + i +
                           " on thread: " + Thread.currentThread().getName());
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {}
    });

    // Tasks may run truly in parallel on different cores
}

// Sequential execution for comparison
public static void sequentialExample() {
    for (int i = 0; i < 4; i++) {
        System.out.println("Sequential task " + i +
                           " on thread: " + Thread.currentThread().getName());
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {}
    }
    // Total time: ~4 seconds
}

public static void main(String[] args) {
    System.out.println("Available processors: " +
                       Runtime.getRuntime().availableProcessors());

    System.out.println("\n--- Sequential ---");
    long start = System.currentTimeMillis();
    sequentialExample();
}

```

```
        System.out.println("Time: " + (System.currentTimeMillis() - start) + "ms");

        System.out.println("\n--- Concurrent ---");
        start = System.currentTimeMillis();
        concurrentExample();
        try { Thread.sleep(3000); } catch (InterruptedException e) {}
        System.out.println("Time: ~2000ms (4 tasks on 2 threads)");

        System.out.println("\n--- Parallel ---");
        start = System.currentTimeMillis();
        parallelExample();
        System.out.println("Time: " + (System.currentTimeMillis() - start) +
            "ms (depends on core count)");
    }
}
```

Practical implications: Use concurrency for I/O-bound tasks where waiting time can be utilized, use parallelism for CPU-bound tasks that benefit from simultaneous computation, concurrency doesn't always improve performance on single-core systems, and parallelism has overhead (thread creation, context switching, synchronization) that may negate benefits for small tasks.

Basic Concurrency Utilities

Q.16 What is the Executor framework? Why is it preferred over manually creating threads?

The **Executor framework** is a high-level concurrency API introduced in Java 5 (`java.util.concurrent` package) that provides a standardized way to manage thread execution. It decouples task submission from the mechanics of how each task will be run, including thread creation, scheduling, and lifecycle management.

Why it's preferred over manual thread creation:

1. **Thread Reusability:** Thread pools reuse existing threads instead of creating new ones for each task, reducing overhead
2. **Resource Management:** Controls the number of concurrent threads, preventing resource exhaustion
3. **Better Performance:** Eliminates thread creation/destruction overhead
4. **Task Queue Management:** Built-in queue management for pending tasks
5. **Lifecycle Management:** Provides proper shutdown mechanisms
6. **Separation of Concerns:** Separates task submission from execution policy
7. **Enhanced Features:** Provides Future, Callable, scheduled execution, and more
8. **Exception Handling:** Better handling of uncaught exceptions
9. **Monitoring and Statistics:** Easier to monitor thread pool metrics

```
import java.util.concurrent.*;

public class ExecutorFrameworkExample {

    public static void manualThreadApproach() {
        // Manual thread creation - NOT recommended
        for (int i = 0; i < 100; i++) {
            final int taskId = i;
            Thread thread = new Thread(() -> {
                System.out.println("Manual thread " + taskId);
            });
            thread.start();
            // Creates 100 threads! Resource intensive, poor performance
        }
    }

    public static void executorApproach() {
        // Executor framework - RECOMMENDED
        ExecutorService executor = Executors.newFixedThreadPool(10);
    }
}
```

```

        for (int i = 0; i < 100; i++) {
            final int taskId = i;
            executor.submit(() -> {
                System.out.println("Task " + taskId + " on " +
                    Thread.currentThread().getName());
            });
            // Reuses pool of 10 threads for 100 tasks
        }

        executor.shutdown(); // Proper cleanup
    }

    public static void main(String[] args) {
        System.out.println("--- Manual Thread Creation ---");
        manualThreadApproach();

        try { Thread.sleep(2000); } catch (InterruptedException e) {}

        System.out.println("\n--- Executor Framework ---");
        executorApproach();
    }
}

```

Core Executor interfaces:

ExecutorService extends Executor and provides lifecycle management (shutdown, shutdownNow, awaitTermination) and task submission with Future return values.

ScheduledExecutorService extends ExecutorService for delayed and periodic task execution.

Q.17 Explain the difference between Callable and Runnable interfaces.

Runnable and **Callable** both represent tasks that can be executed by threads, but they have key differences:

Runnable:

- Functional interface with run() method
- Returns void (no result)
- Cannot throw checked exceptions
- Available since Java 1.0
- Used with Thread class and ExecutorService

Callable:

- Functional interface with call() method

- Returns a result of generic type V
- Can throw checked exceptions
- Available since Java 5
- Used primarily with ExecutorService, returns Future

```
import java.util.concurrent.*;

public class CallableVsRunnableExample {

    static class RunnableTask implements Runnable {
        public void run() {
            System.out.println("Runnable on: " + Thread.currentThread().getName());
            // No return value, no checked exceptions
        }
    }

    static class CallableTask implements Callable<Integer> {
        public Integer call() throws Exception {
            System.out.println("Callable on: " + Thread.currentThread().getName());
            return 42; // Returns value, can throw exceptions
        }
    }

    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        // Runnable - returns null
        Future<?> runnableFuture = executor.submit(new RunnableTask());
        System.out.println("Runnable result: " + runnableFuture.get());

        // Callable - returns actual value
        Future<Integer> callableFuture = executor.submit(new CallableTask());
        System.out.println("Callable result: " + callableFuture.get());

        // Lambda examples
        executor.submit(() -> System.out.println("Runnable lambda"));
        Future<String> future = executor.submit(() -> "Callable lambda result");
        System.out.println(future.get());

        executor.shutdown();
    }
}
```

When to use which:

Use **Runnable** when you don't need a return value, for fire-and-forget tasks, or when working with Thread class directly.

Use **Callable** when you need to return a result from the task, when you need to throw checked exceptions, or when you want to check task completion status.

Q.18 What is a Future in Java? How do you use it to get results from asynchronous tasks?

Future represents the result of an asynchronous computation. It provides methods to check if the computation is complete, wait for its completion, and retrieve the result. It's a placeholder for a value that will be available in the future.

Key methods:

- **get():** Waits if necessary for computation to complete, then retrieves result
- **get(timeout, unit):** Waits for at most the given time for result
- **cancel(mayInterruptIfRunning):** Attempts to cancel execution
- **isCancelled():** Returns true if task was cancelled
- **isDone():** Returns true if task completed (normally, exceptionally, or cancelled)

Basic Future:

```
ExecutorService ex = Executors.newSingleThreadExecutor();

Future<Integer> f = ex.submit(() -> {
    Thread.sleep(1000);
    return 100;
});

System.out.println(f.get()); // Blocks
ex.shutdown();
```

Future with Timeout:

```
ExecutorService ex = Executors.newSingleThreadExecutor();

Future<String> f = ex.submit(() -> {
    Thread.sleep(5000);
    return "Done";
});

try {
    System.out.println(f.get(2, TimeUnit.SECONDS));
} catch (TimeoutException e) {
    f.cancel(true);
}

ex.shutdown();
```

Future limitations:

Cannot be explicitly completed, no callback mechanism (blocking get only), cannot chain multiple futures easily, and limited composition capabilities.

Modern alternative: `CompletableFuture` (Java 8+) addresses these limitations with non-blocking callbacks, composition, and exception handling.

Q.19 Describe the different types of thread pools available in `Executors` class.

The **`Executors`** factory class provides several convenient methods to create different types of thread pools, each optimized for specific use cases:

- 1. `newFixedThreadPool(int nThreads)`:** Creates a pool with fixed number of threads. Reuses threads from a shared unbounded queue. Best for known, limited concurrency.
- 2. `newCachedThreadPool()`:** Creates threads as needed, reuses idle threads. Threads timeout after 60 seconds of inactivity. Best for many short-lived asynchronous tasks.
- 3. `newSingleThreadExecutor()`:** Single worker thread with unbounded queue. Guarantees sequential execution. Best for tasks that must execute sequentially.
- 4. `newScheduledThreadPool(int corePoolSize)`:** Fixed-size pool for delayed and periodic tasks. Best for scheduling recurring tasks.
- 5. `newWorkStealingPool(int parallelism)`:** Uses work-stealing algorithm (`ForkJoinPool`). Best for parallel processing and divide-and-conquer tasks.
- 6. `newSingleThreadScheduledExecutor()`:** Single thread for scheduled tasks. Best for single background scheduled task.

Examples:**1. Fixed Thread Pool (Limited Threads)**

```
ExecutorService ex = Executors.newFixedThreadPool(3);

for (int i = 1; i <= 5; i++) {
    int n = i;
    ex.submit(() ->
        System.out.println("Task " + n + " - " +
            Thread.currentThread().getName())
    );
}

ex.shutdown();
```

Uses **fixed number of threads**. Extra tasks wait in queue.

2. Scheduled Thread Pool (Delayed/Periodic Tasks)

```
ScheduledExecutorService sch =  
    Executors.newScheduledThreadPool(2);  
  
sch.schedule(() ->  
    System.out.println("Runs after 2 sec"),  
    2, TimeUnit.SECONDS  
);  
  
sch.shutdown();
```

Choosing the right pool:

Fixed: Known, predictable workload with bounded resource requirements

Cached: Many short-lived tasks with unpredictable volume

Single: Sequential task execution requirement or single background processor

Scheduled: Recurring or delayed task execution

Work Stealing: CPU-intensive parallel tasks (ForkJoinPool based)

Q.20 What is ThreadLocal? When would you use it and what are its potential pitfalls?

ThreadLocal provides thread-local variables where each thread has its own independently initialized copy of the variable. Each thread accessing the variable gets and sets its own isolated copy.

When to use ThreadLocal:

1. User session management in web applications
2. Database connection per thread
3. Storing request context in servlet applications
4. SimpleDateFormat (not thread-safe) per thread
5. Maintaining transaction context
6. Performance optimization (avoiding synchronization)

Potential pitfalls:

1. **Memory leaks:** Especially in thread pool environments where threads are reused

2. **Inheritance issues:** Child threads don't inherit parent's ThreadLocal values (unless using InheritableThreadLocal)
3. **Testing difficulty:** Hidden dependencies make testing harder
4. **Overuse:** Can lead to hard-to-debug issues and poor design
5. **Thread pool reuse:** Values persist when threads are reused

Basic ThreadLocal (Per-Thread Data)

```
ThreadLocal<Integer> tl =
    ThreadLocal.withInitial(() -> 1);

Runnable task = () -> {
    tl.set(tl.get() + 10);
    System.out.println(
        Thread.currentThread().getName() +
        " = " + tl.get()
    );
};

new Thread(task).start();
new Thread(task).start();
```

Q.21 What is the difference between execute() and submit() methods in ExecutorService?

Both **execute()** and **submit()** are used to submit tasks to an ExecutorService, but they have important differences:

execute(Runnable):

- Defined in Executor interface
- Accepts only Runnable
- Returns void (no return value)
- Cannot track task completion
- Exceptions thrown in task are handled by UncaughtExceptionHandler
- Fire-and-forget semantics

submit():

- Defined in ExecutorService interface
- Three overloaded versions: submit(Runnable), submit(Runnable, T result), submit(Callable)

- Returns Future object
- Can track task completion and get results
- Exceptions are captured in Future, available via get()
- Allows checking status and cancellation

Use execute() when:

- You don't need task result
- You don't need to track completion
- Fire-and-forget semantics are acceptable
- Task is Runnable (not Callable)

Use submit() when:

- You need to get task result
- You need to track task completion status
- You want to handle exceptions properly
- You might need to cancel the task
- You're using Callable

Q.22 Explain CountdownLatch. Provide a use case where it would be beneficial.

CountDownLatch is a synchronization aid that allows one or more threads to wait until a set of operations being performed by other threads completes. It's initialized with a count and threads can wait until this count reaches zero by calling `countDown()`.

Key characteristics:

- One-time use (cannot be reset)
- Thread-safe countdown mechanism
- Threads block on `await()` until count reaches zero
- Multiple threads can wait on same latch
- Count can be decremented by any thread

Use cases:

1. Waiting for multiple services to start before proceeding
2. Ensuring all worker threads complete before processing results
3. Parallel test execution coordination
4. Game start synchronization (wait for all players ready)
5. Batch processing coordination


```
import java.util.concurrent.*;

public class LatchExample {

    public static void main(String[] args) throws Exception {

        CountdownLatch latch = new CountdownLatch(3); // 3 tasks

        ExecutorService ex = Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 3; i++) {
            int n = i;

            ex.submit(() -> {
                System.out.println("Task " + n + " started");

                try { Thread.sleep(1000); } catch (Exception e) {}

                System.out.println("Task " + n + " done");
                latch.countDown(); // Reduce count
            });
        }

        latch.await(); // Main thread waits here

        System.out.println("All tasks finished!");

        ex.shutdown();
    }
}
```

How It Works

1. new CountdownLatch(3) → sets counter to 3
2. Each task calls countDown() → reduces counter
3. await() → waits until counter becomes 0
4. When count = 0 → waiting thread continues

Key methods:

- **await():** Waits until count reaches zero
- **await(timeout, unit):** Waits with timeout
- **countDown():** Decrements the count
- **getCount():** Returns current count

Important notes: CountdownLatch cannot be reused (use CyclicBarrier for reusable barrier), count cannot be increased after initialization, and countDown() doesn't block (only await() blocks).

Q.23 What is a CyclicBarrier? How does it differ from CountdownLatch?

CyclicBarrier is a synchronization aid that allows a set of threads to wait for each other to reach a common barrier point. Unlike `CountDownLatch`, it can be reused after the waiting threads are released.

Key differences from `CountDownLatch`:

CountDownLatch: One-time use, count cannot be reset, any thread can call `countDown()`, waiting and counting are separate operations, used when one or more threads wait for others to complete.

CyclicBarrier: Reusable (cyclic), automatically resets after barrier is broken, only waiting threads count towards barrier, threads wait and count simultaneously (`await()`), used when threads must wait for each other at barrier point, can trigger action when barrier is reached.

```
import java.util.concurrent.*;

public class BarrierExample {

    public static void main(String[] args) {

        CyclicBarrier barrier =
            new CyclicBarrier(3,
                () -> System.out.println("All threads reached barrier!")
            );

        ExecutorService ex = Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 3; i++) {
            int n = i;

            ex.submit(() -> {
                System.out.println("Task " + n + " working");

                try { Thread.sleep(1000); } catch (Exception e) {}

                System.out.println("Task " + n + " waiting");
                try {
                    barrier.await(); // Wait here
                } catch (Exception e) {}

                System.out.println("Task " + n + " resumed");
            });
        }

        ex.shutdown();
    }
}
```

How It Works

1. new CyclicBarrier(3) → waits for 3 threads
2. Each thread calls await()
3. All threads block until 3 arrive
4. Then all are released together

When to use CyclicBarrier:

- Parallel algorithm phases where threads must synchronize at each phase
- Iterative computations where threads work together in rounds
- Multi-player game synchronization
- Parallel testing scenarios with multiple test phases

When to use CountdownLatch:

- One-time event waiting (service startup)
- Coordinating start of multiple threads
- Waiting for completion of multiple tasks
- Simple countdown scenarios

Q.24 What is a Semaphore? Explain with an example use case.

Semaphore is a synchronization construct that maintains a set of permits. Threads acquire permits before proceeding and release them when done. It's used to control access to a limited resource pool.

Types:

1. **Binary Semaphore:** Permits = 1 (similar to mutex)
2. **Counting Semaphore:** Permits > 1 (resource pooling)

Key characteristics:

- Controls concurrent access to shared resources
- Maintains count of available permits
- Blocks threads when no permits available
- Optionally fair (FIFO) or unfair
- Unlike locks, permits can be released by different threads

Common use cases:

1. Database connection pooling
2. Limiting concurrent API calls
3. Rate limiting
4. Bounded resource access (printers, file handles)
5. Implementing producer-consumer with capacity limits

```
import java.util.concurrent.*;

public class SemaphoreExample {

    public static void main(String[] args) {

        Semaphore semaphore = new Semaphore(2); // Allow 2 threads only

        ExecutorService ex = Executors.newFixedThreadPool(5);

        for (int i = 1; i <= 5; i++) {
            int n = i;

            ex.submit(() -> {
                try {
                    semaphore.acquire(); // Get permit

                    System.out.println("Task " + n + " entered");
                    Thread.sleep(1000);

                    System.out.println("Task " + n + " leaving");

                } catch (Exception e) {
                } finally {
                    semaphore.release(); // Return permit
                }
            });
        }

        ex.shutdown();
    }
}
```

How It Works

1. new Semaphore(2) → allows only 2 threads
2. acquire() → waits for permit
3. release() → gives back permit

4. Extra threads wait until permit is free

Key methods:

- **acquire():** Acquires permit, blocks if unavailable
- **acquire(n):** Acquires n permits
- **tryAcquire():** Tries to acquire without blocking
- **tryAcquire(timeout, unit):** Tries with timeout
- **release():** Releases permit
- **availablePermits():** Returns available permits

Q.25 What are the common thread-safe collections in Java? Name at least five.

Java provides numerous thread-safe collection classes in the **java.util.concurrent** package that handle synchronization internally, eliminating the need for external synchronization.

Common thread-safe collections:

1. **ConcurrentHashMap** - Thread-safe hash map with better performance than Hashtable
2. **CopyOnWriteArrayList** - Thread-safe list for read-heavy scenarios
3. **CopyOnWriteArraySet** - Thread-safe set based on CopyOnWriteArrayList
4. **ConcurrentLinkedQueue** - Non-blocking thread-safe queue
5. **LinkedBlockingQueue** - Blocking queue with optional capacity
6. **ArrayBlockingQueue** - Fixed-size blocking queue
7. **PriorityBlockingQueue** - Unbounded blocking priority queue
8. **ConcurrentSkipListMap** - Sorted thread-safe map
9. **ConcurrentSkipListSet** - Sorted thread-safe set
10. **ConcurrentLinkedDeque** - Thread-safe double-ended queue
11. **SynchronousQueue** - Zero-capacity blocking queue
12. **DelayQueue** - Delayed element blocking queue

Key characteristics:

Lock-free algorithms: Many use CAS (Compare-And-Swap) operations for better performance

Weakly consistent iterators: Iteration may not reflect recent modifications but never throw `ConcurrentModificationException`

Atomic operations: Built-in atomic compound operations like `putIfAbsent`, `compute`, `merge`

No external synchronization needed: Internal synchronization is handled by the collection

Q.26 Explain ConcurrentHashMap. How does it achieve better performance than Hashtable?

ConcurrentHashMap is a thread-safe, high-performance hash table implementation that provides better concurrency than Hashtable through segment-level locking (Java 7) or CAS operations with synchronized nodes (Java 8+).

Key differences from Hashtable:

Hashtable: Uses single lock for entire table, all operations block each other, poor concurrent performance, and iteration requires copying or external synchronization.

ConcurrentHashMap: Uses fine-grained locking or lock-free algorithms, multiple concurrent reads without blocking, writes block only affected segments/buckets, lock-free reads in Java 8+, and weakly consistent iterators don't require locking.

Performance improvements:

1. **Concurrent reads:** Multiple threads read simultaneously without blocking
2. **Segment-level locking (Java 7):** Different segments can be modified concurrently
3. **CAS operations (Java 8+):** Lock-free updates using atomic compare-and-swap
4. **Better scalability:** Performance scales with number of processors
5. **No null keys/values:** Allows simpler, faster code paths

When to use ConcurrentHashMap:

High concurrent read/write scenarios, when null keys/values are not needed, when iteration doesn't need precise snapshot, and for general-purpose concurrent map operations.

When NOT to use:

Need for null keys/values (use Collections.synchronizedMap instead), require strict iteration consistency, or single-threaded scenarios (HashMap is faster).

Q.27 What is CopyOnWriteArrayList? When should you use it?

CopyOnWriteArrayList is a thread-safe variant of ArrayList where all mutative operations (add, set, remove) create a fresh copy of the underlying array. This makes it ideal for scenarios with frequent reads and infrequent writes.

Key characteristics:

1. **Thread-safe without synchronization:** Reads require no locking

2. **Snapshot iterators:** Iteration uses snapshot at time of iterator creation
3. **No ConcurrentModificationException:** Iterators never fail
4. **Memory overhead:** Each modification creates new array copy
5. **Write performance:** Slower writes due to array copying
6. **Read performance:** Very fast reads (no synchronization)

When to use:

- Read operations vastly outnumber writes (90%+ reads)
- List is small to moderate in size
- Event listener lists
- Observer pattern implementations
- Configuration data that rarely changes
- Scenarios where iteration consistency is important

When NOT to use:

- Frequent modifications
- Large lists
- Performance-critical write operations
- Memory-constrained environments

CopyOnWriteArraySet: Similar implementation for Set interface, backed by CopyOnWriteArrayList, same characteristics and use cases.

Alternatives to consider:

For frequent writes: `ConcurrentHashMap.newKeySet()` or `Collections.synchronizedList()` For large lists: `ConcurrentSkipListSet` or other concurrent collections For specific patterns: Custom synchronization or immutable collections

Q.28 What is the difference between BlockingQueue and regular Queue?

Regular Queue (e.g., LinkedList, ArrayDeque):

- Non-blocking operations
- `add()` throws exception if queue is full
- `poll()` returns null if queue is empty
- No built-in thread safety (except `ConcurrentLinkedQueue`)
- No waiting mechanism for producers/consumers

BlockingQueue (e.g., ArrayBlockingQueue, LinkedBlockingQueue):

- Thread-safe by design
- Blocking operations when queue is full or empty
- put() blocks if queue is full (waits for space)
- take() blocks if queue is empty (waits for elements)
- Supports timeouts with offer() and poll()
- Ideal for producer-consumer patterns

Common BlockingQueue Implementations:

- **ArrayBlockingQueue** - Bounded, backed by array
- **LinkedBlockingQueue** - Optionally bounded, backed by linked nodes
- **PriorityBlockingQueue** - Unbounded, ordered by priority
- **SynchronousQueue** - Zero capacity, direct handoff
- **DelayQueue** - Elements available after delay

Q.29 What is an AtomicInteger? How does it differ from using synchronized on an integer?

AtomicInteger is a thread-safe integer class that supports atomic operations without using locks, leveraging low-level CPU compare-and-swap (CAS) instructions.

Key Differences:

AtomicInteger:

- Lock-free using CAS operations
- Better performance under contention
- Non-blocking (optimistic locking)
- Single variable only
- Lower overhead for simple operations
- Supports atomic compound operations

synchronized:

- Uses monitor locks
- Blocking (pessimistic locking)
- Can protect multiple variables
- Higher overhead

- Simpler for complex operations

```
// Using synchronized - locks involved
class SynchronizedCounter {
    private int count = 0;

    public synchronized void increment() {
        count++; // Lock acquired, incremented, lock released
    }

    public synchronized int get() {
        return count;
    }
}

// Using AtomicInteger - lock-free
class AtomicCounter {
    private AtomicInteger count = new AtomicInteger(0);

    public void increment() {
        count.incrementAndGet(); // CAS operation, no lock
    }

    public int get() {
        return count.get();
    }
}
```

When to Use Each:

Use AtomicInteger when:

- Simple counter or flag
- High read/write concurrency
- Single variable updates
- Performance is critical

Use synchronized when:

- Multiple related variables need updating together
- Complex operations beyond increment/decrement
- Need guaranteed visibility across operations
- Code clarity is more important than performance

Other Atomic Classes:

```
AtomicLong      // For long values
AtomicBoolean   // For boolean flags
AtomicReference // For object references

// Example with AtomicReference
AtomicReference<String> status = new AtomicReference<>("PENDING");
status.compareAndSet("PENDING", "RUNNING");

// Array versions
AtomicIntegerArray
AtomicLongArray
AtomicReferenceArray
```

Advanced Concurrency Patterns

Q.30 Explain the happens-before relationship in the Java Memory Model. Why is it important?

The **happens-before** relationship is a fundamental ordering guarantee in the Java Memory Model (JMM) that defines when actions in one thread are guaranteed to be visible to another thread.

Core Concept: If action A happens-before action B, then all memory writes made by A are visible to B. This prevents instruction reordering that could break multithreaded programs.

Key Happens-Before Rules:

- **Program Order Rule:** Each action in a thread happens-before every subsequent action in that same thread
- **Monitor Lock Rule:** Unlocking a monitor happens-before every subsequent lock of that same monitor
- **Volatile Variable Rule:** A write to a volatile field happens-before every subsequent read of that field
- **Thread Start Rule:** A call to `Thread.start()` happens-before any action in the started thread
- **Thread Termination Rule:** Any action in a thread happens-before another thread detects that thread has terminated
- **Transitivity:** If A happens-before B, and B happens-before C, then A happens-before C

Why It's Important: Without happens-before, the JMM allows aggressive optimizations like caching values in registers or reordering instructions. This could lead to threads seeing stale or inconsistent data. The happens-before relationship provides the minimum synchronization guarantees needed for correct concurrent programs while still allowing compiler and hardware optimizations that don't violate these rules.

Q.31 What is the difference between optimistic and pessimistic locking? Provide examples of each in Java.

Pessimistic Locking: Assumes conflicts are likely, so it locks resources immediately to prevent concurrent access.

```
// Example: Traditional synchronized approach
public class BankAccount {
    private double balance;
```

```

public synchronized void withdraw(double amount) {
    // Lock acquired - no other thread can access until done
    if (balance >= amount) {
        balance -= amount;
    }
}
}

```

Optimistic Locking: Assumes conflicts are rare, so it doesn't lock resources. Instead, it checks before committing whether the data was modified by another thread.

```

// Example: Using AtomicReference with CAS
public class BankAccount {
    private AtomicReference<Double> balance = new AtomicReference<>(1000.0);

    public void withdraw(double amount) {
        while (true) {
            Double current = balance.get();
            if (current < amount) break;

            Double newBalance = current - amount;
            // Only update if no one else changed it
            if (balance.compareAndSet(current, newBalance)) {
                break; // Success
            }
            // Retry if CAS failed (someone else modified it)
        }
    }
}

```

Trade-offs: Pessimistic locking is simpler but can cause thread blocking and reduced concurrency. Optimistic locking offers better performance under low contention but can waste CPU cycles retrying under high contention.

Q.32 Describe the Fork/Join framework. When would you use it over traditional thread pools?

The **Fork/Join framework** is designed for parallel divide-and-conquer algorithms that recursively split tasks into smaller subtasks.

Core Components:

- ForkJoinPool: Special thread pool optimized for fork/join tasks
- RecursiveTask<V>: Returns a result
- RecursiveAction: Doesn't return a result

```

class SumTask extends RecursiveTask<Long> {
    private long[] array;
    private int start, end;
    private static final int THRESHOLD = 1000;

    protected Long compute() {
        if (end - start <= THRESHOLD) {
            // Base case: compute directly
            return computeDirectly();
        } else {
            // Recursive case: split task
            int mid = (start + end) / 2;
            SumTask left = new SumTask(array, start, mid);
            SumTask right = new SumTask(array, mid, end);

            left.fork(); // Async execution
            long rightResult = right.compute(); // Current thread computes
            long leftResult = left.join(); // Wait for forked task

            return leftResult + rightResult;
        }
    }
}

```

When to Use Over Thread Pools: Use Fork/Join when you have recursive decomposition problems where tasks spawn subtasks dynamically. Traditional thread pools are better for independent, non-recursive tasks. Fork/Join's work-stealing scheduler is optimized for scenarios where tasks create other tasks, preventing thread starvation.

Q.33 What is work stealing in the context of Fork/Join? How does it improve performance?

Concept: Each thread in a ForkJoinPool has its own deque (double-ended queue) of tasks. When a thread finishes its tasks, it "steals" work from the tail of another thread's deque.

How It Works:

- Thread executes its own tasks from the head of its deque (LIFO order for cache locality)
- Idle threads steal from the tail of busy threads' deques (FIFO order to minimize contention)
- This balances load dynamically without central coordination

Performance Benefits:

- **Better load balancing:** Idle threads don't stay idle when work exists elsewhere
- **Reduced contention:** Each thread primarily works on its own queue; stealing is infrequent
- **Cache efficiency:** LIFO execution keeps recently created tasks in cache
- **Scalability:** No central work queue bottleneck

Q.34 Explain the concept of lock striping. How does ConcurrentHashMap use this technique?

Lock Striping divides data into segments, each protected by its own lock, allowing multiple threads to access different segments concurrently.

ConcurrentHashMap Implementation (pre-Java 8):

```
// Conceptual illustration
class ConcurrentHashMap<K,V> {
    // Array of segments, each with its own lock
    final Segment<K,V>[] segments;

    static class Segment<K,V> extends ReentrantLock {
        HashEntry<K,V>[] table;

        V put(K key, V value) {
            lock(); // Lock only this segment
            try {
                // Modify only this segment's data
            } finally {
                unlock();
            }
        }
    }
}
```

Benefits: If the map has 16 segments, up to 16 threads can modify different segments simultaneously. This dramatically improves concurrency compared to locking the entire map.

Modern Java 8+ Implementation: Uses CAS operations and synchronized blocks on individual nodes rather than segments, providing even finer-grained concurrency.

Q.35 What are reentrant locks? How do they differ from intrinsic locks (synchronized)?

Reentrant Lock: A thread can acquire the same lock multiple times without deadlocking itself. Each acquisition increments a counter; the lock is only released when the counter reaches zero.

```
// Intrinsic lock (synchronized) - automatically reentrant
public synchronized void outer() {
    inner(); // Can call without deadlocking
}

public synchronized void inner() {
    // Same thread can enter even though it holds the lock
}
```

```
// ReentrantLock - explicit reentrant lock
ReentrantLock lock = new ReentrantLock();

public void method() {
    lock.lock();
    try {
        anotherMethod(); // Can reacquire
    } finally {
        lock.unlock();
    }
}

public void anotherMethod() {
    lock.lock();
    try {
        // Same thread, second acquisition
    } finally {
        lock.unlock();
    }
}
```

Key Differences:**Intrinsic Locks (synchronized):**

- Simpler syntax
- Automatic lock release (exception-safe)
- Cannot be interrupted while waiting
- No fairness guarantees
- No try-lock capability

ReentrantLock:

- More flexible API
- Can be interrupted: `lockInterruptibly()`
- Support for fairness: `new ReentrantLock(true)`
- Try-lock with timeout: `tryLock(100, TimeUnit.MILLISECONDS)`
- Can query lock state: `isLocked()`, `getHoldCount()`
- Requires explicit unlock in finally block

Q.36 Describe read-write locks. When would ReentrantReadWriteLock be more efficient than synchronized?

Concept: Allows multiple readers OR one writer, based on the observation that reads don't conflict with each other.

```
ReadWriteLock rwLock = new ReentrantReadWriteLock();
Lock readLock = rwLock.readLock();
Lock writeLock = rwLock.writeLock();

// Multiple threads can execute this concurrently
public Data read() {
    readLock.lock();
    try {
        return data;
    } finally {
        readLock.unlock();
    }
}

// Only one thread can execute this, blocks all readers
public void write(Data newData) {
    writeLock.lock();
    try {
        data = newData;
    } finally {
        writeLock.unlock();
    }
}
```

When More Efficient Than Synchronized:

- Read-heavy workloads (reads >> writes)
- Long-duration read operations
- When read contention is high

With synchronized, even multiple readers block each other. With ReentrantReadWriteLock, they execute concurrently, significantly improving throughput.

Not Beneficial When:

- Write-heavy workloads
- Very short read operations (overhead of more complex locking outweighs benefits)

Q.37 What is a StampedLock? How does it improve upon ReentrantReadWriteLock?

Advanced locking mechanism introduced in Java 8 that improves upon ReentrantReadWriteLock with optimistic reads.

Three Modes:

1. **Write Lock:** Exclusive access
2. **Read Lock:** Shared access (like ReentrantReadWriteLock)
3. **Optimistic Read:** No actual lock, just validation

```
StampedLock sl = new StampedLock();

// Optimistic read - the key improvement
public Data optimisticRead() {
    long stamp = sl.tryOptimisticRead(); // No lock acquired!
    Data currentData = data; // Read data

    if (!sl.validate(stamp)) {
        // Data might have been modified, upgrade to read lock
        stamp = sl.readLock();
        try {
            currentData = data;
        } finally {
            sl.unlockRead(stamp);
        }
    }
    return currentData;
}

// Traditional read lock
public Data read() {
    long stamp = sl.readLock();
    try {
        return data;
    } finally {
        sl.unlockRead(stamp);
    }
}

// Write lock
public void write(Data newData) {
    long stamp = sl.writeLock();
    try {
        data = newData;
    } finally {
        sl.unlockWrite(stamp);
    }
}
```

Improvements Over ReentrantReadWriteLock:

- **Optimistic reads:** In read-dominated scenarios with rare writes, threads can read without acquiring locks, validating afterward
- **Lock conversion:** Can upgrade/downgrade locks
- **Better performance:** Less overhead, no support for reentrancy (simpler implementation)
- **Not reentrant:** Simpler, faster, but requires care to avoid deadlock

Trade-offs: Not reentrant, more complex API, and optimistic reads must handle potential validation failures.

Q.38 Explain the ABA problem in concurrent programming. How would you detect or prevent it?

The Problem: In concurrent programming with CAS operations, a value changes from A to B and back to A. A thread using CAS thinks nothing changed, even though intermediate modifications occurred.

```
// Thread 1 reads value A
AtomicReference<Node> head = new AtomicReference<>(nodeA);
Node observed = head.get(); // Sees A

// Thread 2: changes A -> B -> A
head.set(nodeB);
head.set(nodeA); // Back to A, but potentially different object!

// Thread 1: CAS succeeds, but state actually changed
head.compareAndSet(observed, newNode); // Success! But missed B
```

Real-World Impact: In data structures like stacks or linked lists, the same memory address might be reused for different nodes. The ABA problem can cause corruption when recycling nodes.

Solutions:**1. Version/Stamp Tagging:**

```
AtomicStampedReference<Node> head = new AtomicStampedReference<>(nodeA, 0);

int[] stampHolder = new int[1];
Node observed = head.get(stampHolder);
int stamp = stampHolder[0];

// CAS with both value AND stamp
head.compareAndSet(observed, newNode, stamp, stamp + 1);
```

2. AtomicMarkableReference: Uses a boolean mark instead of an integer stamp, useful when you only need to track whether a change occurred.

3. Garbage Collection: In garbage-collected languages like Java, if you hold a reference to an object, it won't be reused, reducing ABA risk (though not eliminating it entirely for primitive values).

Q.39 What is compare-and-swap (CAS)? How do atomic classes use it?

Definition: An atomic operation that updates a value only if it matches an expected value. It's a hardware-level primitive that enables lock-free programming.

Pseudocode:

```
boolean compareAndSwap(address, expected, new) {
    atomic {
        if (*address == expected) {
            *address = new;
            return true;
        }
        return false;
    }
}
```

How Atomic Classes Use CAS:

```
public class AtomicInteger {
    private volatile int value;

    public final int incrementAndGet() {
        for (;;) {
            int current = get();
            int next = current + 1;
            if (compareAndSet(current, next)) {
                return next;
            }
            // Retry if CAS failed
        }
    }

    // Uses sun.misc.Unsafe for actual CAS operation
    public final boolean compareAndSet(int expect, int update) {
        return unsafe.compareAndSwapInt(this, valueOffset, expect, update);
    }
}
```

Benefits:

- No locks needed (non-blocking)
- No deadlock risk
- Better performance under low-to-moderate contention

- Thread-safe without context switching

Challenges:

- ABA problem
- Can waste CPU under high contention (spinning)
- More complex to implement correctly

Q.40 Describe the concept of lock-free and wait-free algorithms. Provide an example.

Lock-Free: At least one thread makes progress in a finite number of steps, even if other threads are delayed. The system as a whole makes progress, though individual threads might starve.

```
// Lock-free stack using CAS
public class LockFreeStack<T> {
    private AtomicReference<Node<T>> head = new AtomicReference<>();

    private static class Node<T> {
        final T value;
        final Node<T> next;
        Node(T value, Node<T> next) {
            this.value = value;
            this.next = next;
        }
    }

    public void push(T value) {
        Node<T> newHead = new Node<>(value, null);
        while (true) {
            Node<T> currentHead = head.get();
            newHead.next = currentHead;
            if (head.compareAndSet(currentHead, newHead)) {
                return; // Success - system progressed
            }
            // Retry - this thread didn't progress, but another did
        }
    }
}
```

Wait-Free: Every thread makes progress in a finite number of steps, regardless of other threads. Stronger guarantee than lock-free.

```
// Wait-free read (simpler example)
public class WaitFreeCounter {
    private final AtomicInteger[] counters; // One per thread

    // Each thread writes to its own counter - no contention
    public void increment(int threadId) {
        counters[threadId].incrementAndGet(); // Always completes
    }

    // Sum all counters for total
    public int get() {
        int sum = 0;
        for (AtomicInteger counter : counters) {
            sum += counter.get();
        }
        return sum;
    }
}
```

Comparison:

- **Blocking (with locks):** Threads can wait indefinitely
- **Lock-Free:** System makes progress, individual threads might retry
- **Wait-Free:** Every thread makes progress (hardest to implement)

Q.41 Explain how `CompletableFuture` improves upon `Future`. Provide examples of its advanced usage.

Future Limitations:

```
Future<String> future = executor.submit(() -> "result");
String result = future.get(); // Blocking! Can't compose or chain
```

CompletableFuture Improvements:

1. Manual Completion:

```
CompletableFuture<String> cf = new CompletableFuture<>();
// Later, from any thread:
cf.complete("result");
```

2. Asynchronous Callbacks:

```
CompletableFuture.supplyAsync(() -> fetchData())
    .thenApply(data -> transform(data))
    .thenAccept(result -> save(result))
    .exceptionally(ex -> handleError(ex));
```

3. Combining Multiple Futures:

```
CompletableFuture<String> cf1 = CompletableFuture.supplyAsync(() -> "Hello");
CompletableFuture<String> cf2 = CompletableFuture.supplyAsync(() -> "World");

// Combine results
CompletableFuture<String> combined = cf1.thenCombine(cf2,
    (s1, s2) -> s1 + " " + s2);

// Wait for all
CompletableFuture.allOf(cf1, cf2).join();

// Wait for any
CompletableFuture.anyOf(cf1, cf2).join();
```

4. Chaining and Composition:

```
CompletableFuture.supplyAsync(() -> getUserId())
    .thenCompose(id -> CompletableFuture.supplyAsync(() -> fetchUser(id)))
    .thenCompose(user -> CompletableFuture.supplyAsync(() -> getOrders(user)))
    .thenAccept(orders -> displayOrders(orders));
```

Advanced Features:

- Non-blocking transformations
- Exception handling with `handle()`, `whenComplete()`
- Timeout support: `orTimeout()`, `completeOnTimeout()`
- Custom executors for different stages

Q.42 What is reactive programming? How does it relate to concurrency in Java?

Reactive Programming is a paradigm centered on asynchronous data streams and the propagation of change. It handles concurrency through non-blocking, event-driven models.

Core Principles:

- **Responsive:** System responds quickly
- **Resilient:** Stays responsive despite failures
- **Elastic:** Scales with load
- **Message-driven:** Asynchronous message passing

Relationship to Java Concurrency:

1. Non-Blocking I/O: Traditional threading blocks on I/O. Reactive approaches use event loops (like Node.js) or reactive streams to handle thousands of concurrent operations with fewer threads.

2. Reactive Streams (Java 9+):

```
// Flow API (Java 9+)
SubmissionPublisher<String> publisher = new SubmissionPublisher<>();

Flow.Subscriber<String> subscriber = new Flow.Subscriber<>() {
    private Flow.Subscription subscription;

    public void onSubscribe(Flow.Subscription subscription) {
        this.subscription = subscription;
        subscription.request(1); // Request backpressure
    }

    public void onNext(String item) {
        processItem(item);
        subscription.request(1); // Request next
    }

    public void onError(Throwable throwable) {
        handleError(throwable);
    }

    public void onComplete() {
        cleanup();
    }
};

publisher.subscribe(subscriber);
```

3. Libraries (Project Reactor, RxJava):

```
// Project Reactor example
Flux.range(1, 100)
    .parallel()
    .runOn(Schedulers.parallel())
    .map(this::expensiveOperation)
    .subscribe(result -> handleResult(result));
```

Benefits for Concurrency:

- Better resource utilization (fewer threads handle more work)
- Backpressure handling prevents overwhelming slow consumers
- Declarative concurrency (transformations automatically parallelized)
- Natural fit for event-driven systems

Q.43 What tools would you use to detect and diagnose deadlocks in a production Java application?

JDK Built-in Tools:

- **jstack** - Generates thread dumps, automatically detects deadlocks
- **jconsole** - GUI monitoring tool with "Detect Deadlock" button
- **jvisualvm** - Visual profiler with thread analysis
- **Java Mission Control (JMC)** - Advanced profiling and monitoring

Programmatic Detection:

- **ThreadMXBean.findDeadlockedThreads()** - JMX-based detection
- **Thread.getAllStackTraces()** - Manual analysis

APM Tools:

- New Relic, Dynatrace, AppDynamics - Production monitoring with alerts

Q.44 What is the overhead of synchronization? How would you measure it in your application?

Synchronization Overhead Includes:

1. **Acquiring/releasing locks** - Even uncontended locks have cost
2. **Memory barriers** - Ensuring visibility across cores
3. **Cache coherence traffic** - Keeping caches synchronized
4. **Context switching** - When threads block

Typical Costs:

- Uncontended lock: ~25-50 nanoseconds

- Contended lock: Microseconds to milliseconds (waiting time)
- Memory barrier: ~10 nanoseconds

Profiling Tools:

- JMH (Java Microbenchmark Harness) - Gold standard for benchmarking
- JProfiler, YourKit - Show lock contention
- async-profiler - Low overhead profiling

Q.45 Explain how garbage collection can impact multi-threaded application performance.

GC Impact on Multi-threaded Apps:

1. Stop-the-World Pauses:

- All application threads pause during GC
- Longer pause = more threads affected
- Can cause thread pool starvation

2. Thread Synchronization:

- GC needs to find all object references
- Requires threads at safe points
- Can delay thread operations

3. Memory Allocation:

- Thread-local allocation buffers (TLABs)
- Contention during TLAB refill
- Young generation pressure from many threads

Mitigation Strategies:

```
// 1. Use appropriate GC
-XX:+UseG1GC           // Good for large heaps
-XX:+UseZGC             // Ultra-low latency (Java 11+)
-XX:+UseShenandoahGC    // Low pause times

// 2. Tune heap sizes
-Xms4g -Xmx4g           // Same size prevents resizing
-XX:NewRatio=2           // Young/Old ratio

// 3. Monitor GC
-XX:+PrintGCDetails
-XX:+PrintGCDateStamps
-Xlog:gc*                // Java 9+

// 4. Reduce object allocation
// Object pooling for frequently created objects
private static final ThreadLocal<StringBuilder> builderPool =
    ThreadLocal.withInitial(() -> new StringBuilder(256));
```

Best Practices:

- Monitor GC logs regularly
- Size heap appropriately
- Minimize object creation in hot paths
- Use concurrent/low-pause GC for latency-sensitive apps
- Consider off-heap storage for large datasets

Q.46 What is livelock? How does it differ from deadlock and how do you resolve it?

Livelock is when threads actively respond to each other but make no progress. Unlike deadlock (threads blocked), livelock threads keep running but repeat the same actions indefinitely.

Differences:**Deadlock:**

- Threads are blocked/waiting
- Thread state: BLOCKED
- No CPU usage
- Circular wait for resources

Livelock:

- Threads are active (RUNNABLE)
- Continuous CPU usage
- Threads respond to each other but no progress
- Similar to people trying to pass each other in a hallway

Example Scenario: Two threads both trying to be "polite" - when detecting conflict, both back off and retry, continuously:

```
// Livelock example
class Resource {
    private Thread owner;

    synchronized boolean tryAcquire(Thread thread) {
        if (owner == null) {
            owner = thread;
            return true;
        }
        return false;
    }

    synchronized void release() {
        owner = null;
    }
}

// Both threads do this:
while (!resource.tryAcquire(Thread.currentThread())) {
    resource.release(); // Be polite, release
    Thread.sleep(10);   // Wait
    // Keep retrying - LIVELOCK!
}
```

Resolving Livelock:

1. Add randomization to retry
2. Implement timeout and give up
3. Use priority or ordering
4. Use higher-level concurrency utilities

```
// Solution: Random backoff
Random random = new Random();
while (!resource.tryAcquire(Thread.currentThread())) {
    Thread.sleep(random.nextInt(100)); // Random wait
}

// Solution: Timeout
```

```
if (!resource.tryAcquire(Thread.currentThread(), 1, TimeUnit.SECONDS)) {
    throw new TimeoutException();
}
```

Q.47 How would you implement a timeout mechanism for acquiring locks to prevent indefinite blocking?

Using tryLock() with timeout:

```
import java.util.concurrent.locks.*;
import java.util.concurrent.*;

// ReentrantLock with timeout
ReentrantLock lock = new ReentrantLock();

if (lock.tryLock(5, TimeUnit.SECONDS)) {
    try {
        // Critical section
    } finally {
        lock.unlock();
    }
} else {
    // Handle timeout - couldn't acquire lock
    throw new TimeoutException("Failed to acquire lock");
}

// ReadWriteLock with timeout
ReadWriteLock rwLock = new ReentrantReadWriteLock();

if (rwLock.writeLock().tryLock(3, TimeUnit.SECONDS)) {
    try {
        // Write operation
    } finally {
        rwLock.writeLock().unlock();
    }
} else {
    // Timeout handling
}
```